

Birla Institute of Technology & Science, Pilani Hyderabad Campus**CS F372: Operating Systems**

Assigned: 24/08/2026**Date of Submission: 04/09/2026****Total Marks: 12****Weightage: 6%**

General Instructions:

- This assignment can be done in groups of **maximum 4** students.
- All your programs must run on an **Ubuntu 24.X virtual machine on Oracle Virtual Box**.
- Recommended allocation: 4 cores, 50 GB virtual disk, 4GB RAM or more on VM.
- Submit all source files and executables along with a readme that has your group details (Names and ID numbers of all members of the group.)
- Put all your deliverables in a zip/tar file and submit through the course webpage (assignment section). Pl make only One submission per group from any members' ID.
- Every assignment will have a demo and viva. The schedule of demo and viva will be intimated later through a separate notice. Demo will be on your laptop.

For any assignment related queries, you may contact both instructors (Prof Geetha and Prof Hota) or the following teaching assistants:

- Arpit Kudre <f20230158@hyderabad.bits-pilani.ac.in>
 - Parth Munjal <f20230058@hyderabad.bits-pilani.ac.in>
-

Problem 1: Write a C program that forks a child process **[2 Marks]**

The parent process has an integer array containing 'n' distinct positive integers, say {18, 24, 35, 49, 10, 63, 27, 40, 14, 21}, where n is even.

Loop Behavior:

- The parent process randomly selects two distinct elements from the array, say **x** and **y**, and **removes them from the array** as they are picked (i.e. no element is ever picked twice). It prints **x** and **y**, then it sends both to the child process via an unnamed Pipe.
- The child process receives **x** and **y**, computes **g**, their GCD, and prints **x**, **y**, and **g**.
- The child then sleeps for **time(NULL)%g milliseconds** (i.e. UNIX_time % g).
- The child sends '**g**' back to the parent via a second unnamed pipe.
- The parent receives **g**, prints it, and sleeps for **g milliseconds**.

Process Termination:

- The process should stop once the array is exhausted (i.e., after **n/2** rounds), or
- after an outside interrupt (like Ctrl+C) is received.

Do not use any global variables.

(Page 1 of 6)

Problem 2: Weighted Resource Monitor**[3 Marks]**

A resource monitoring system is a utility program that continuously observes system resource consumption and provides mechanisms to manage processes based on a combined usage metric, serving as a practical tool for system administrators to prioritize which processes are worth investigating first in case of serious breach. This dual requirement, automated surveillance combined with human decision-making, is elegantly addressed through a parent-child process architecture, with the added realism of coordinating that decision-making over a persistent, kernel-managed message queue rather than a simple pipe.

Write a C program mimicking a weighted resource monitor behavior:

The program should fork a child process which would be responsible for monitoring resource usage and killing selected processes, while the parent would handle user interaction. The parent and child shall coordinate control messages using a **System V message queues** as discussed in the class.

Input Variables:

The program takes 3 integers n , k , r as input from the user. Assume n , k , r to be more than 0.

- n : time in seconds between 2 consecutive prints
- k : processes displayed in each print
- r : iterations after which a process is killed

Ranking Formula:

Instead of ranking by a single metric, each process's usage score is computed as:

$$\text{usage_score} = 3 * \text{CPU\%} + 2 * \text{MEM\%}$$

where $\text{CPU\%}$ and $\text{MEM\%}$ are the process's CPU and memory usage percentages (obtainable via `ps` with BSD syntax). The k processes with the highest `usage_score` should be printed each cycle.

Loop Behaviour:

- The child process prints the top k processes by `usage_score` every n seconds.
- The output must be printed in descending order of `usage_score`, and must display the PID, command name, CPU%, MEM%, and the computed `usage_score` for each process.

PID	COMMAND	CPU%	MEM%	SCORE
-----	---------	------	------	-------

- You may use `pipe`, `fork`, `exec/exec1p`, `dup2`, and external utilities such as `ps`, `awk`, `sort`, and `head` as needed.

Parent-Child Coordination:

- A System V message queue must be created **before** the `fork ()` call, so that both parent and child can access the same queue.
- After `r` iterations:
 1. The child process waits for a message from the parent.
 2. The parent prompts the user to enter the PID of the process to act on.
 3. The parent sends this input to the child as a message on the queue.
- Upon receiving the message, the child does the following, in order:
 1. If the input is `-2`, the child removes the message queue and exits the program.
 2. If the input is `-1`, the child skips any action and resumes printing.
 3. Otherwise, the child first prints a short description of that process, its command name, owner, current CPU%, MEM%, and `usage_score` and only then kills the process. The child prints a confirmation after killing it, and resumes printing the monitored list.

Program Termination:

- The program terminates only when the user inputs `-2`, or after an outside interrupt (like Ctrl+C) is received.

Do not use any global variables. Use `ps` with BSD syntax to obtain CPU and memory usage, and format output appropriately.

Problem 3: Log Frequency Analyzer**[2 Marks]**

Server log files often contain repeated entries — for example, the same IP address hitting an endpoint multiple times. A common task in system administration is identifying which values occur most frequently in a given column of such a file, so that repeated or anomalous activity can be spotted quickly.

Example Command – Running the required chain of Unix utilities manually on a file `access.log`, to find the most frequent values in column 1:

```
bash$ cut -d' ' -f1 access.log | sort | uniq -c | sort -rn | head -5
```

Sample Output:

```
42 192.168.1.10
31 192.168.1.14
19 10.0.0.5
12 10.0.0.9
7 172.16.0.3
```

Where each line shows the count followed by the value, sorted in descending order of frequency, limited to the top 5.

Now write a program `logtop` which takes a log file & a column number as arguments and prints the top 5 most frequent values in that column with their counts, in the format shown above.

(Page 3 of 6)

Example Command – Running `logtop`:

```
max@Parths-MacBook-Pro Downloads % gcc -Wall -o logtop logtop.c
[./logtop access.log 1
 5 192.168.1.10
 4 192.168.1.14
 3 10.0.0.5
 2 10.0.0.9
 1 172.16.0.3
```

[NOTE: You are NOT supposed to implement counting, sorting, or frequency-aggregation logic in your C code. You are instead expected to use UNIX commands such as `cut`, `sort`, `uniq` and `head`, and get this desired output through constructs you learnt in this course such as forks, pipes, `exec` etc., and let your C code route their outputs.] If the file has fewer than 5 distinct values, print however many are available. You can assume that the maximum line length in the log file will be 512 bytes.

Problem 4: Developing a Shell of your own

[3 Marks]

The shell or command interpreter is the fundamental user interface to an Operating System. Every shell is structured as the following loop:

1. Print out a prompt
2. Read a line of input from the user
3. Parse the line into the program name, and an array of parameters
4. Use the `fork()` system call to spawn a new child process
 - a) The child process then uses the `exec()` system call to launch the specified program
 - b) The parent process (the shell) uses the `wait()` system call to wait for child to terminate
5. When the child (i.e. the launched program) finishes, the shell repeats the loop by jumping to step 1.

Write a simple shell in C used to interact with a warehouse conveyor-belt controller, named `belt_shell`, that has the following properties:

Internal Commands: These are the commands directly executed by shell process (no need to fork):

- `add_item <name>` — Adds `<name>` to the belt's item queue (store in a local fixed-size array of at most 10 items). If the queue is already full, show an error message. If `<name>` is missing, show an error message.
- `list_items` — Prints every item currently in the queue, one per line, in the order they were added. If the queue is empty, print `Queue is empty`.
- `quit` — allows us to exit the shell.

External Commands: These are the commands the Shell has to fork for; the child process calls `exec()` while the parent waits for the child to complete execution (as detailed in the loop above):

- `date` — displays the current system date/time.
- `ping <address>` — tries to ping the given address exactly 4 times (e.g. address: `google.com`).

Additional Requirements:

- The shell prompt should be: `belt-control$`
- The item queue starts empty.
- Show an error message for commands not defined above.
- You should handle the `SIGINT` signal (Ctrl+C). During normal operation, if the user presses Ctrl+C, the shell should not terminate. Instead, it should print: `[ALERT] Emergency stop triggered, item queue cleared`, clear the queue back to empty, continue normal operation — only the `quit` command can exit the shell.
- Assume maximum input in a line is 1023 characters.

[Hint: To split the command and the arguments you can look into `strtok` function man page.]

Sample Test:

```
max@Parths-MacBook-Pro Downloads % gcc -Wall -o belt_shell belt_shell.c
./belt_shell
belt-control$ add_item box1
belt-control$ add_item box2
belt-control$ list_items
box1
box2
belt-control$ date
Tue Aug 18 18:03:15 IST 2026
belt-control$ quit
```

Problem 5: Implementing & Testing a Custom System Call in the Linux Kernel**[2 Marks]**

A system call is a programmatic interface that allows applications to request services from the operating system's kernel, acting as a controlled gateway between user programs and privileged system resources as discussed in class. When a program needs to perform operations it cannot do directly — like reading files, creating processes, or accessing hardware, it makes a system call that triggers the CPU to switch from user mode to kernel mode, allowing the OS to safely execute the requested operation.

The procedure to create a system call varies in different versions of Linux. When you add a system call you will have to recompile the Linux kernel, so you must have the Linux source tree installed on your system.

Adding a System Call to the Linux Kernel:

Add a system call to the Linux kernel (version: 6.14), called `setnice_logged`, which takes a single integer argument `nice_val` (range -20 to 19). Its functionality:

- Changes the nice value (scheduling priority) of the calling process to `nice_val`.
- Validates that `nice_val` is within the legal range before applying it; if invalid, returns an error (`-EINVAL`) without modifying anything.
- On a successful change, prints the calling process's PID, its name (`comm`), the old nice value, and the new nice value to the kernel buffer.

Use the following function to test your system call. Here `setnice_logged()` is a local wrapper to a syscall you define in the kernel before compiling it.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#define __NR_setnice_logged 470 // replace with the syscall number you assigned
int setnice_logged(int nice_val) {
    return syscall(__NR_setnice_logged, nice_val);
}
int main() {
    int ret = setnice_logged(5);

    if (ret == 0)
        printf("Nice value successfully changed.\n");
    else
        perror("setnice_logged failed");

    return 0;
}
```

To test if the message was in the kernel buffer run the command: `dmesg | grep setnice_logged`

Helpful Resources:

- [\[linux-source-tree\]](#)
- [\[task struct priority fields – linux kernel 6.14\]](#)
- [\[nice value to kernel priority mapping\]](#)
- [\[standard kernel error codes – EINVAL and friends\]](#)
- [\[printing to kernel buffer – linux kernel 6.14\]](#)
- [\[obj-y in make files\]](#)
- [\[syscall tables and headers\]](#)
- [\[fork new method – linux kernel 6.14\]](#)
- [\[fork exact implementation\]](#)(Line 2681-2694)

Important Instructions: Please submit your deliverables by midnight on 6th September 2026 via the Google Classroom page. Retain a copy of your submitted code, as you will need to present it later. Every assignment will include a demo and viva session, **during which each team member will be randomly asked to modify (add or remove) a feature from any of the given problems and must solve it on the spot over the VM you must have created during the Tutorial classes.**